Copy a `Derived` into a `Base` and the Derived part is cut off.

That is object slicing.

```
Derived d;
Base b = d;   // d's y is gone
```

It compiles without a single warning.

swipe →

What gets lost

```
struct Base {
    int x = 1;
    virtual void print() const;   // "Base"
};
struct Derived : Base {
    int y = 2;
    void print() const override;     // "Derived"
};
```

OBJECT	HOLDS
`d`	`x` and `y`
`b` (copy)	`x` **only**

Even the virtual call changes

```
Derived d;
Base b = d;
b.print();
```

Prints "Base", not "Derived".

`b` is a real `Base` object. Its hidden vtable pointer was built for `Base`.

A virtual call only follows the real type if you go through a pointer or reference to the original object.

Where it hides

```
void f(Base b);                    // slices

std::vector<Base> v;
v.push_back(Derived());             // slices

catch (Base e) { }                    // slices
```

Passing by value, storing in a container, catching by value.

All three sliced in my test, and the virtual call then printed "Base".

The fixes

```
void f(const Base& b);

std::vector<std::unique_ptr<Base>> v;

catch (const Base& e) { }
```

Pass by reference, store pointers, catch by reference.

Or make the copy impossible: `Base(const Base&) = delete;` turns the slice into a compile error. I checked.

The rule

IF IT IS POLYMORPHIC	THEN
a parameter	`const Base&`
a container element	`unique_ptr<Base>`
a copy	a `clone()` function

Never copy a polymorphic object by value.

Is there a `vector<Base>` hiding in your code?